

## 单元二：算法与算法方法

### 第五章 算法设计方法（子程序）



## 5.3 子程序概念及设计

### 5.4 C语言的函数

### 5.5 头文件

### 5.6 函数应用举例

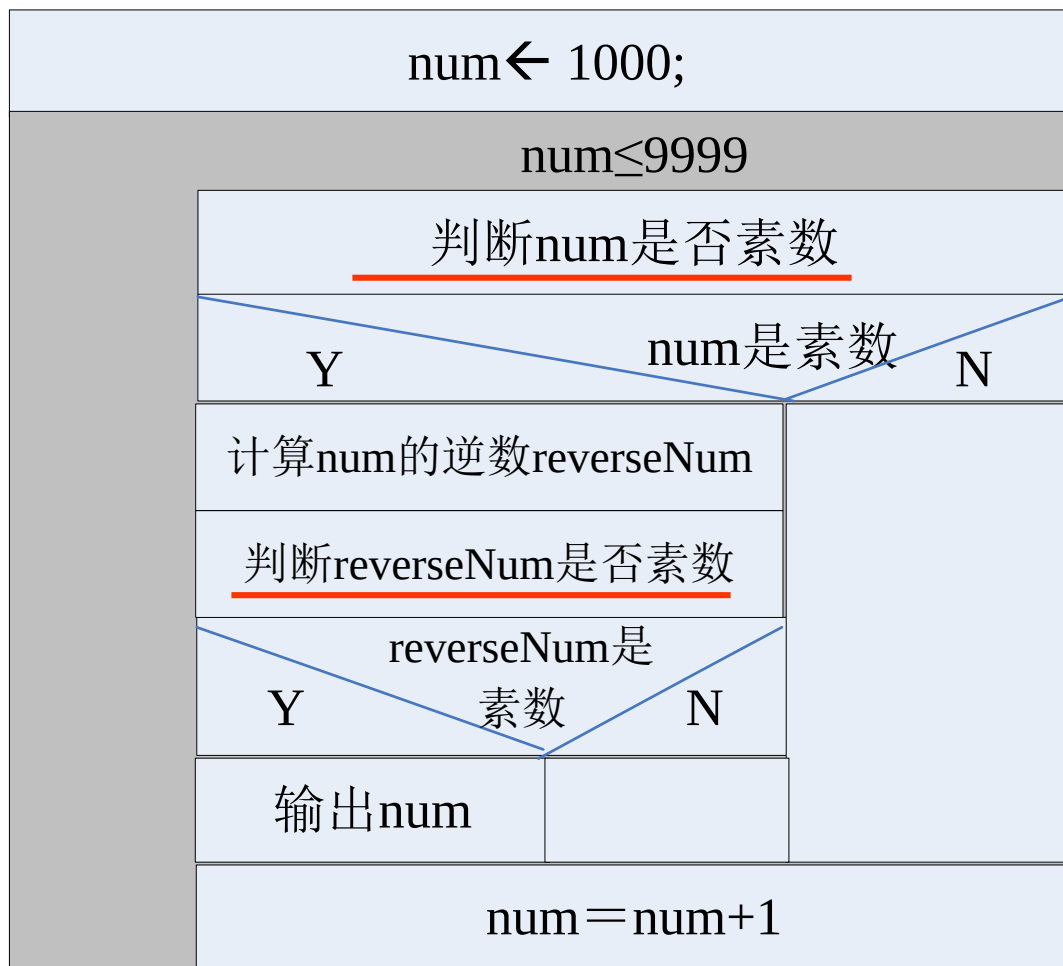


- 编写程序，求所有四位可逆素数，所谓可逆素数是这么一种素数，它的逆数也是素数。

包含的主要功能：

- 判断一个数是否素数。
- 求一个整数的逆数。如1234的逆数是4321。

## 5.3 子程序概念与设计



求可逆素数

- 本程序中判断素数的代码会出现两次;
- 判断素数、求整数逆数这两个功能是独立的功能,且在多个程序中都有可能用到:
  - 求一个整数的逆数: 该功能在判断一个整数是否是回文数中也被用到;
  - 判断一个数是否是素数: 该功能在对整数进行素数分解中用到。

## 5.3 子程序概念与设计

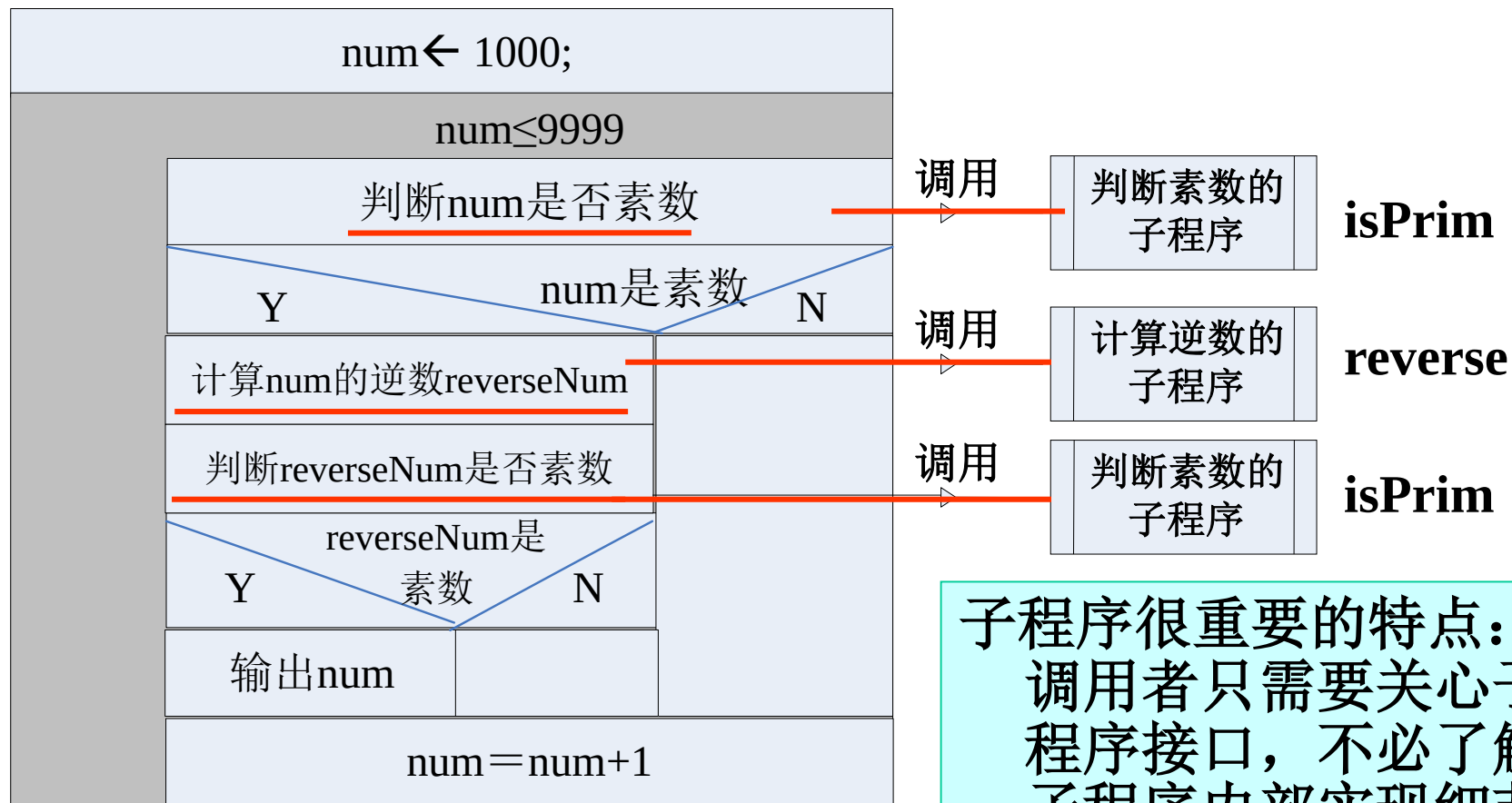
- 能否将完成上述独立功能的代码包装成一个单元，并且可以供其他代码来调用？--答案是可以使用**子程序**

### 一. 子程序的定义

子程序是封装并给以命名的一段程序代码，这段程序代码完成子程序所定义的功能，可供调用。

**封装**：调用者只需要关心代码能完成什么功能，如何调用代码（即子程序接口），而不需要关心代码的内部实现。

## 5.3 子程序概念与设计



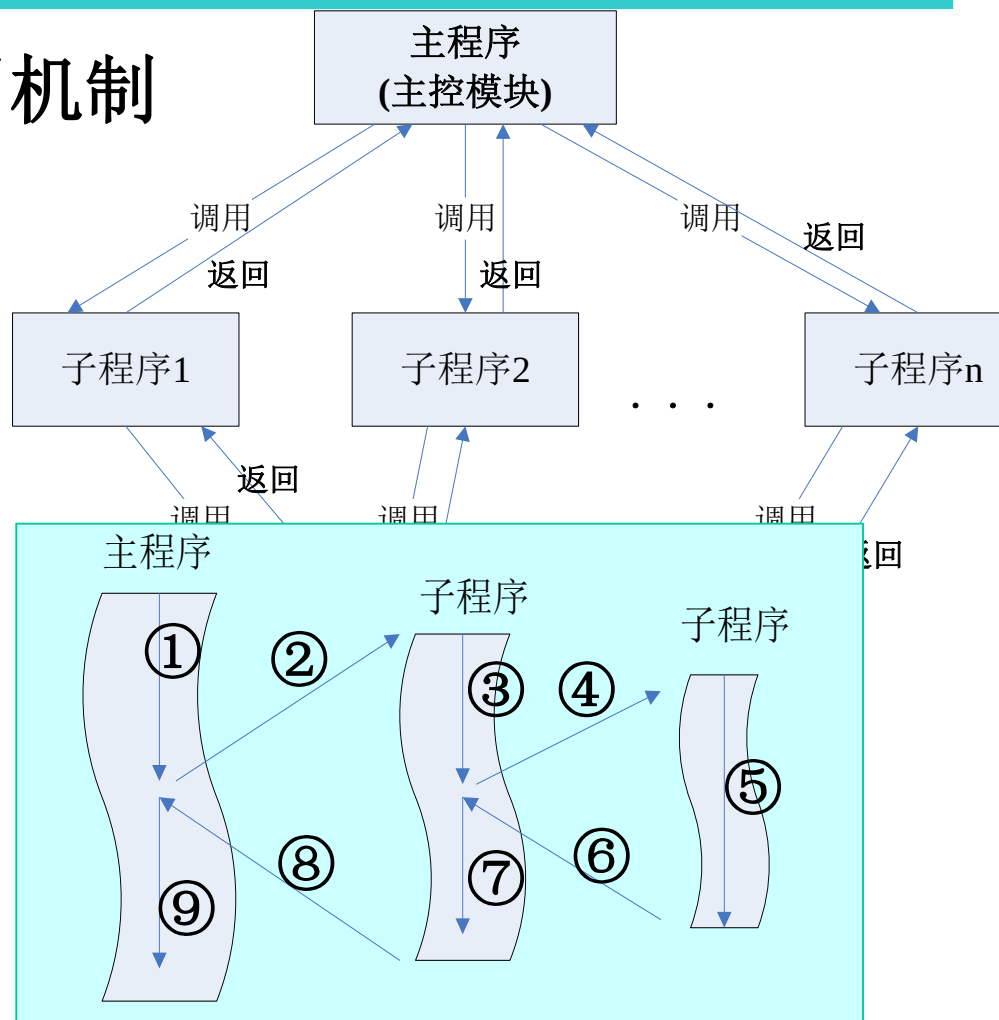
子程序很重要的特点：  
调用者只需要关心子  
程序接口，不必了解  
子程序内部实现细节。

可以设计子程序**isPrim**，用于判断一个整数是否是素数；子程序**reverse**，用于计算一个整数的逆数；

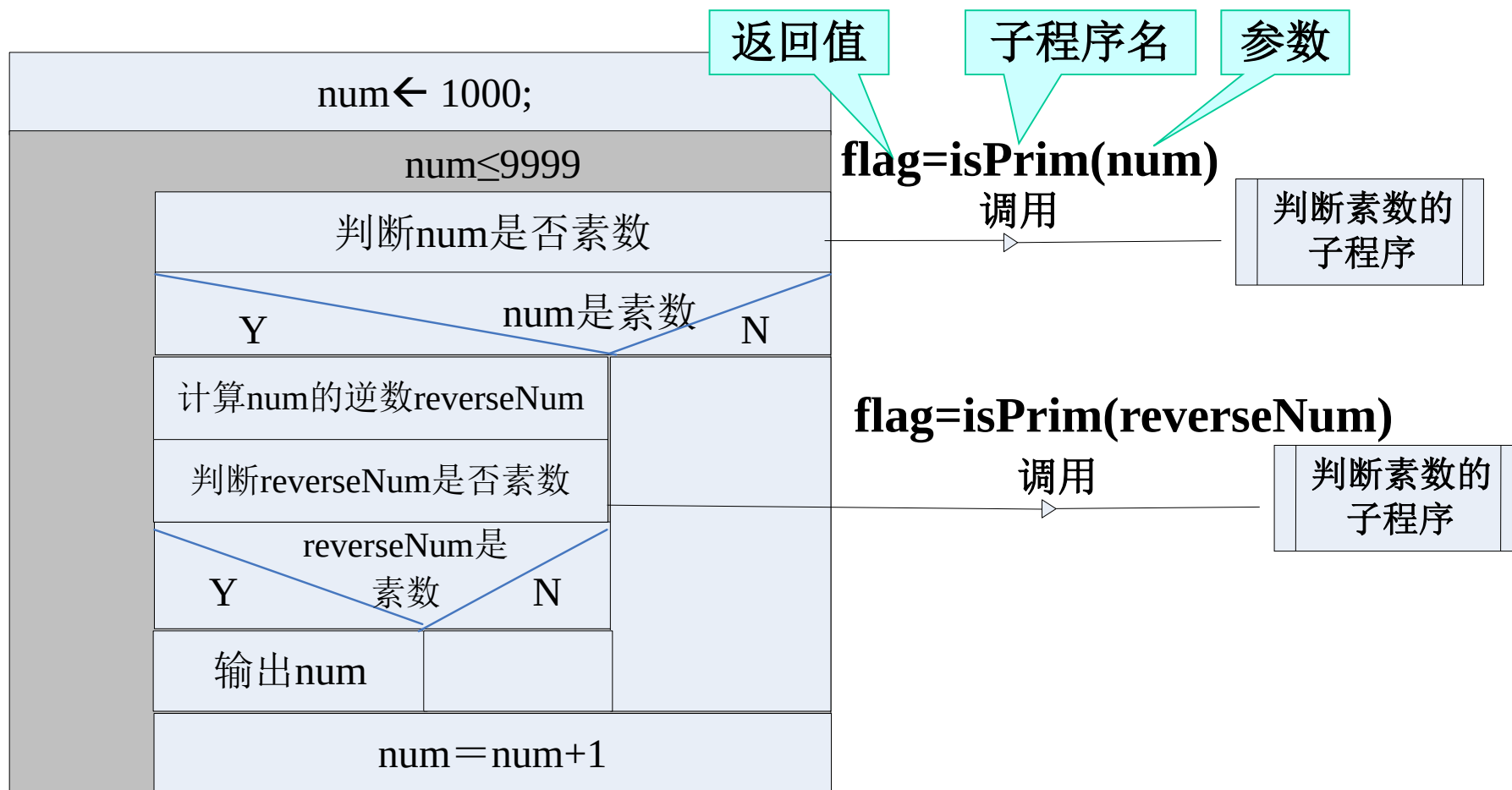
## 5.3 子程序概念与设计

### 二.子程序的控制和调用机制

- ◆通过子程序名进行调用；
- ◆调用时需要传递一些供子程序计算和处理的数据（**参数**）；
- ◆子程序执行完成后需要返回处理结果；
- ◆子程序可以调用其他的子程序；



## 5.3 子程序概念与设计





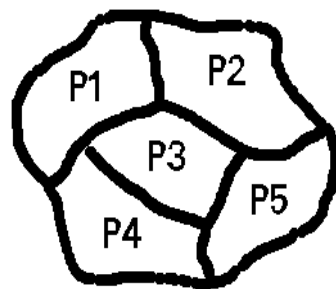
## 5.3 子程序概念与设计

### 三. 引入子程序的目的

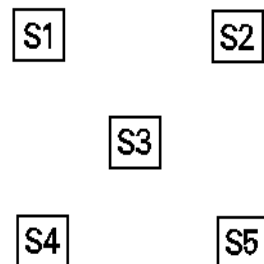
1. 程序“复用”，避免在程序中使用重复代码；
2. 结构化程序设计的需要：

自顶向下、逐步细化，将复杂问题分解为相对简单的子问题，这些子问题用子程序实现，从而提高主程序结构的清晰性和易读性。

3. 使程序的调试和维护变得更加容易。



需要通过软件解决的“问题”



软件的“解决方案”

```
for(num = 1000;num <= 9999;num++){  
    flag = isPrim(num);//是素数则返回1， 否则返回0  
    if (flag == 1){  
        reverseNum = reverse(num);//求逆数  
        flag = isPrim(reverseNum);  
        if (flag == 1){  
            printf("%d\n",num);  
        }  
    }  
}
```

### 四. 子程序设计原则

高内聚：功能相对独立和完整；

低耦合：与外界（调用者）的关系尽量松散，  
不要太紧密，使其能方便地被重用；

需要合理地设计子程序参数和子程序执行的局部环境 来达到以上目标。

### 五. 子程序在C语言中的实现机制

C语言中的函数机制

## 5.3 子程序概念与设计

## 5.4 C语言的函数

### 5.4.1 函数

#### 5.4.2 函数的定义

#### 5.4.3 函数的调用

#### 5.4.4 函数原型

## 5.5 头文件

## 5.6 函数应用举例

C语言中用函数实现子程序设计思想。较大的C语言应用程序，往往是由多个函数组成的（用户自定义函数或标准库函数），每个函数完成明确的功能；

每一个函数应该只完成单一的预定好的任务，并且函数名能有效地反映函数完成的任务；如果不能选择简洁的函数名，那可能函数完成的功能太多，建议拆分成几个较小的函数。

C标准库提供了丰富的函数集，能够完成常用的数学计算、字符串操作、输入/输出等有用操作，程序员可以直接使用、从而减少工作量；

## 5.3 子程序概念与设计

## 5.4 C语言的函数

### 5.4.1 函数

### 5.4.2 函数的定义

### 5.4.3 函数的调用

### 5.4.4 函数原型

## 5.5 头文件

## 5.6 函数应用举例

## 5.4.2 函数的定义

- 函数设计的要求：
  - 明确该函数的功能（函数应该只完成单一的任务）；
  - 定义该函数的接口（即函数头，包括函数名、参数和返回值）
  - 定义该函数的功能实现部分

## 5.4.2 函数的定义

### 函数定义的格式:

返回值类型 函数名( 参数列表 ) /\*接口定义部分\*/

#### 5、函数如何返回

有返回值时: `return` 返回值表达式;

无返回值时:

1) `return`;

2) 运行到函数结束的}处

列表定义为`void`; 如 `int print(void)`

(3) 如果不列出参数的类型, 编译器就假定其为`int`类型。但最好明确指定参数的类型, 即使是`int`型, 最好也明确定义。

果不一致, 则以返回值类型为准(进行类型转换)。

函数定义: 求x的y次方

```
int power(int x,int y)
```

```
int i, p;
```

```
p = 1;
```

```
for (i = 1; i <= y; i++)
```

```
    p = p * x;
```

```
return p;
```

```
}
```



### 1、函数名

简洁、能反映出函数的功能。如：square、printf等。

### 2、参数列表

- (1) 参数列表声明了在调用函数时函数所接收的参数，  
形式为：数据类型 参数1[, 数据类型 参数2……]
- (2) 如果函数不接收任何参数，则参数列表定义为void；  
如 int print(void)
- (3) 如果不列出参数的类型，编译器就假定其为int类型。  
但最好明确指定参数的类型，即使是int型，最好也明确定义。

### 3、返回值类型

- (1) 指返回给函数调用者的结果的类型；
- (2) 如果不指明返回值类型，编译器将假定返回值是int型（最好明确指定）；
- (3) 如果函数不返回任何值（即函数功能实现部分无return语句），则返回值类型定义成**void**。若既无return语句，返回值又不是void类型，则函数将返回一个不确定的值；

### 4、返回值与return语句

(1) **return**语句的一般格式:

**return ( 返回值表达式 );**

**或 return 返回值表达式 ;**

(2) **return**语句的功能: 返回调用函数, 并将“返回值表达式”的值带给调用函数;

(3) **return**语句中返回值表达式的类型要和返回值的类型说明一致。如果不一致, 则以返回值类型为准(进行类型转换)。

### 5、函数如何返回

有返回值时：**return** 返回值表达式;

无返回值时:

1) **return;**

2) 运行到函数结束的}处

### 6、函数中的语句能访问的变量

可以访问本函数中定义的参数、变量，但是不能访问其他函数中定义的参数和变量

## 5.4.2 函数的定义

**练习1：**设计一个函数**IsLeapYear(year)**，用于判断**year**年是否是闰年。如果是，则返回**1**；否则返回**0**。

**year**年是否是闰年的判断条件为：

- a) **year**能被**4**整除但不能被**100**整除；
- 或b) **year**能被**400**整除。

## 5.4.2 函数的定义

**/\*函数功能：判断n是否是闰年**

**参数： year ： 要判断的年份**

**返回值：若是闰年，返回1，否则返回0\*/**

```
int isLeapYear(int year)
{
    if ( year % 4 == 0 && year % 100 != 0
        || year % 400 == 0)
        return 1;
    else
        return 0;
}
```

常见的程序设计错误：

- (1) 把同一种类型的参数声明为类似于形式 **f(float x, y)**，而不是**f(float x, float y)**；f是函数
- (2) 在函数内部把函数参数再次定义成局部变量是一种语法错误；如：

```
int sum(int x, int y)
{
    int x, y; //错误！
    return (x+y);
}
```

## 5.4.2 函数的定义

(3) 不能在一个C函数的内部定义另一个函数;

```
int main()
```

```
{
```

```
...
```

```
    int sum(int x,int y)
```

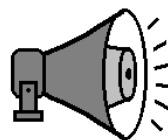
```
    {
```

```
        return(x+y);
```

```
    }
```

```
...
```

```
}
```



不允许!



## 5.3 子程序概念与设计

## 5.4 C语言的函数

### 5.4.1 函数

### 5.4.2 函数的定义

### 5.4.3 函数的调用

### 5.4.4 函数原型

## 5.5 头文件

## 5.6 函数应用举例

## 5.4.3 函数的调用

- 函数是一段封装的代码，能完成预定好的、独立的任务，能被其他函数所调用。
- 定义一个函数就相当于制造了一台数据加工机器，那么，如何来使用这台机器进行数据加工呢？
- 首先我们需要指定使用哪台机器（函数名），还需要给出待加工的数据（实参）。

## 5.4.3 函数的调用

```
int square(int); /*函数原型*/
```

```
int main()
```

```
{  
    int x;  
    for (x = 1; x <= 10; x++)  
        printf("%4d", square(2 * x));  
}
```

实参

形参

函数调用

```
int square(int y) /*函数定义*/
```

```
{  
    return (y * y);  
}
```

函数调用形式:

函数名(实参1,实参2,...);

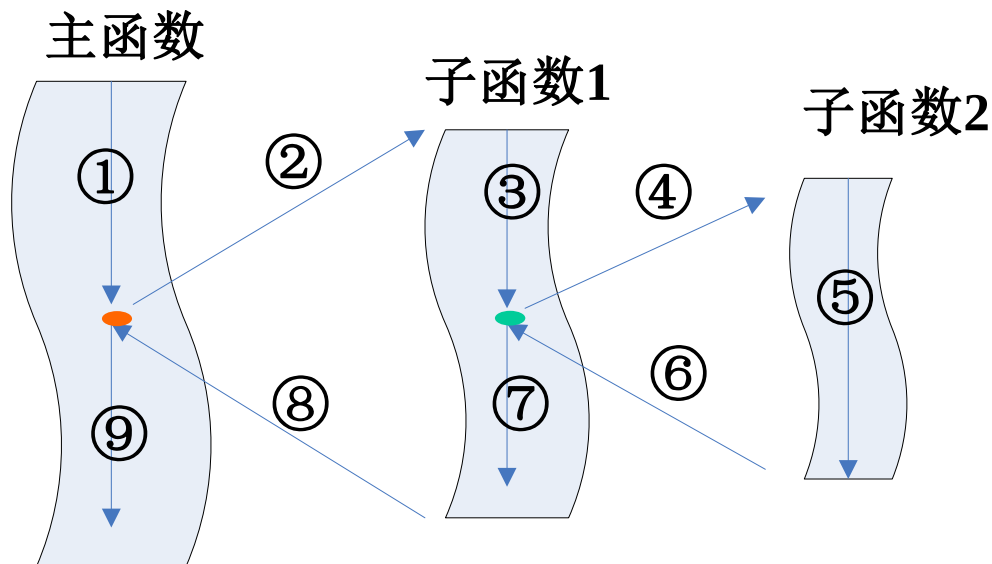
如: `square(2*x)`, 此处`2*x`是实参。  
实参可以是常量、变量、表达式、函数等, 在调用前必须具有确定的值。

调用时, 会为形参分配内存, 然后将实参的值复制一份到形参中。

实参的个数、类型和顺序, 应该与形参个数、类型和顺序一致, 才能正确地进行数据传递。

### 函数调用过程

- 函数的调用和执行的实质是**控制转移**，调用函数时，将控制转到被调用的函数，被调函数执行结束时，则将控制转回主调函数，继续执行后续的操作。



```
int square(int); /*函数原型*/
```

```
int main()
{
    int x;
    for (x = 1; x <= 10; x++)
        printf("%4d",square(2 * x));
    return 0 ;
}
```

函数调用

```
int square(int y) /*函数定义*/
{
    return (y * y);
}
```

## 函数调用过程:

1. 给被调用函数分配存储空间;
2. 计算实际参数表达式的值;
3. 把实际参数的值按赋值转换规则转换成形式参数的类型。
4. 把转换后的实际参数（实参）的值送入形式参数（形参）中。
5. 运行调用函数中的语句;
6. 计算返回值表达式的值，并转换成函数的结果类型;
7. 释放被调用函数占用的存储空间;
8. 带着转换后的值返回调用函数。

## 5.4.3 函数的调用

```
int square(int); /*函数原型*/
```

```
int main()
{
    int x;

    for (x = 1; x <= 10; x++)
        printf("%4d",square(2 * x));
    return 0 ;
}
```

```
int square(int y) /*函数定义*/
{
    return (y * y);
}
```

函数原型的作用：是对被调用函数的**接口声明**，它告诉编译器函数返回的数据类型、函数所要接收的参数个数、参数类型和参数顺序，编译器用函数原型校验函数调用是否正确。

## 5.4.3 函数的调用

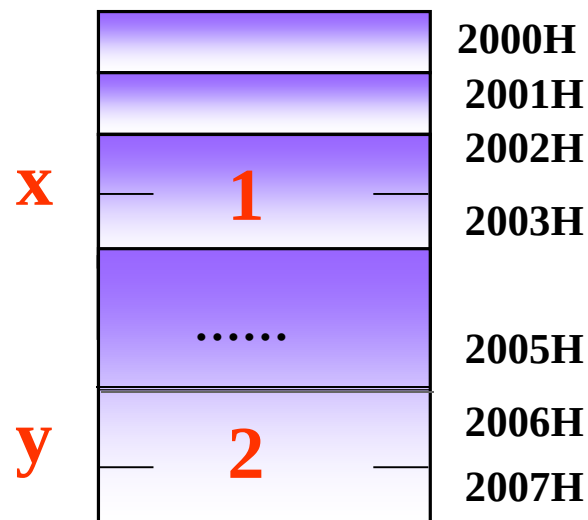
```
int square(int); /*函数原型*/
```

```
int main()
{
    int x;

    → for (x = 1; x <= 10; x++)
    → printf( "%4d" ,square(2 * x));
      return 0 ;
}
```

函数调用

存储空间



```
→ int square(int y) /*函数定义*/
{
    → return(y * y);
}
```

4

在C语言中，可以用以下几种方式调用函数：

对于有返回值的函数：

- (1) **函数表达式**。函数作为表达式的一个操作数，出现在表达式中，此时先进行函数调用，然后函数返回值替代函数调用参与表达式运算。

如：`i = 2*max(x, y); if (isPrim(n))...`

- (2) **函数实参**。函数作为另一个函数调用的实际参数。这种情况是把该函数的返回值作为实参进行传送。如：  
`printf(" the large number is %d", max(x, y));`

对于无返回值或者不需要使用返回值的函数：

- (3) **函数语句**。C语言中的函数可以只进行某些操作而不返回函数值，这时的函数调用可作为一条独立的语句。  
如：`printf("hello\n");`



## 5.4.3 函数的调用

- **切记：**实参的个数、类型和顺序，应该与形参个数、类型和顺序一致，才能正确地进行数据传递。
- 实参可以是常量、变量、表达式、函数等。无论实参是何种类型的量，在进行函数调用时，它们都必须具有确定的值，以便把这些值传送给形参。
- 形参变量只有在被调用时，才分配内存单元；调用结束时，即刻释放所分配的内存单元。

## 5.4.3 函数的调用

- 实参和形参占用不同的内存单元，即使同名也互不影响。
- 实参对形参的数据传送是单向的，即只能把实参的值传送给形参，而不能把形参的值反向传送给实参。
- 不同函数中可以使用相同名称和类型的变量，它们占用不同的内存单元，互不影响。

练习1. 假设函数接口定义为: `int function(int a,int b)`

`result`为整型变量, 请判断下面对函数的调用是否正确:

- 1) `result = int function(int a, int b);`
- 2) `result = function;`
- 3) `fuction(a,b);`

- 练习2：定义以下问题的函数头

设计一函数，求一个正整数的长度；

**int length(int n)**

调用举例：输出变量num中数据的长度

**printf("the length of num is :%d",length(num));**

设计一函数，求三个整数中的最大值；

**int max(int n1, int n2, int n3)**

调用举例：求3个变量的最大值

**maxNum=max(num1,num2,num3)**

**练习3：** 要求设计一个函数： **isPrim(x)**

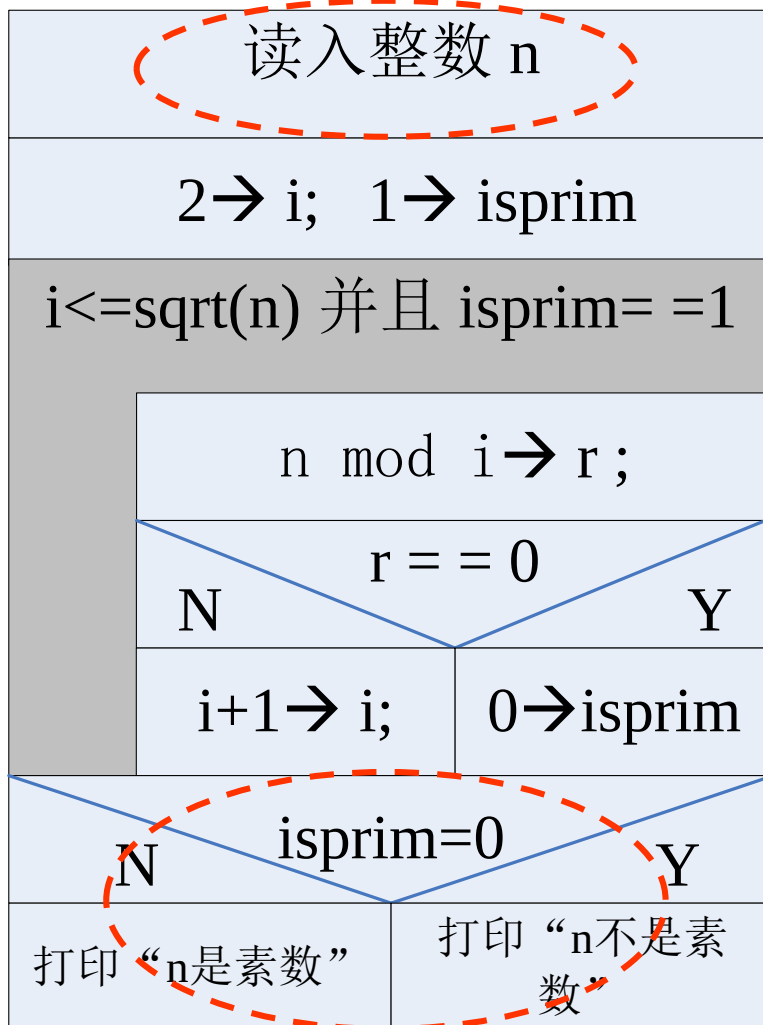
函数定义： **isPrim(x)=1** 当x 是素数；  
              **=0** 当x 不是素数 ；

然后在**main**函数中输出所有长度为4的素数

函数头定义：

```
int isPrim(int x);
```

## 5.4.3 函数的调用



→ 修改：要判断的数通过参数传入

→ 修改：判断结果通过return语句返回



**函数功能：**判断一个正整数是否为素数.若是,则返回**1**;否则返回**0**。

**输入参数：****n**：要判断的整数。

**返回值：**若**n**是素数，则返回值为**1**;否则返回值为**0**。\*/

```
int isPrim(int n)
{
    int i; /*不断判断n能否被i整除。i的取值范围是2~sqrt(n)*/
    int isPrim; /*isPrim=1:表示n是质数； isPrim=0: 表示n不是质数*/

    i = 2;
    isPrim = 1; /*初始设定n是素数。在判断中一旦发现不是素数，则
                isPrim被修改成0。*/

    while (i * i <= n && isPrim == 1){
        if (n % i == 0)
            isPrim = 0;
        else
            i++;
    };

    return isPrim; /*返回*/
}
```

## 5.4.3 函数的调用

```
#include<stdio.h>
int isPrim(int);

int main()
{
    int i;
    for(i = 1000; i <= 9999; i++)
        if (isPrim(i) == 1)
            printf("%d\t", i);
    return 0;
}
int isPrim(int n)
{
    //略
}
```

输出4位正整数中的素数

等价于：

```
for(i = 1000; i <= 9999; i++)
    if(isPrim(i))
        printf("%d\t", i);
```

此种写法更接近于人惯用的  
表达方式



## 5.4.3 函数的调用

//isPrim算法2:

```
int isPrim (int n)
```

```
{
```

```
    int i; /*不断判断n能否被i整除。i的取值范围是2~sqrt(n)*/
```

```
    i = 2;
```

```
    while (i * i <= n && n % i != 0){
```

```
        i++;
```

```
    };
```

```
    if (i * i > n)
```

```
        return 1;
```

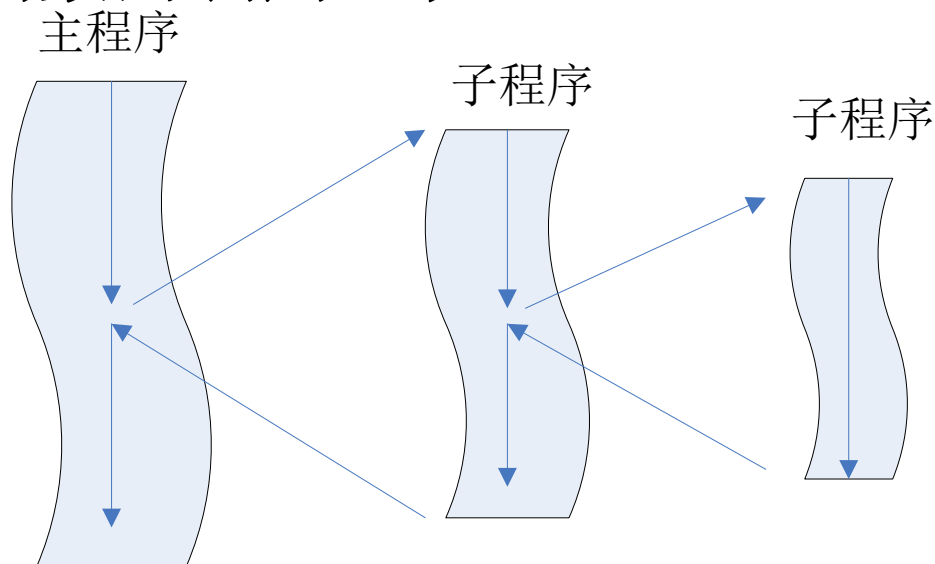
```
    else
```

```
        return 0;
```

```
}
```

## 5.4.3 函数的调用

- 如何确保能够逐层返回到上一级调用？
  - 要想正确地返回，必须知道返回地址。如何获取返回地址？
- 为何不同的函数可以使用同名的参数和变量？
- 进一步剖析函数的调用过程。



```
int num; //全局变量
```

```
void func(int n)
```

```
{
```

```
    static int m; //静态局部变量
```

```
    ...
```

```
}
```



内存

## 5.4.3 函数的调用

**系统区：**用于存放系统软件和运行需要的数据，如操作系统。只要机器一运行，这部分空间就必须保留给系统软件使用

**用户程序代码区：**存放用户程序代码

**静态存储区：**存放程序运行期间不释放的数据（**静态局部变量、全局变量**）

**栈区：**存放程序运行期间会被释放的数据（**函数参数、非静态局部变量**）以及活动的控制信息

**堆区：**用户可以在程序运行过程中根据需要动态地进行存储空间的分配，这样的分配在堆区进行

## 5.4.3 函数的调用一栈区简介

重点介绍和函数调用相关的栈区：

- 一个类比：装东西的箩筐

箩筐：一个存放东西的容器，框底封了口。

装东西：东西从箩筐口入；

取东西：东西从箩筐口出，框顶的东西先被取出。即最先放入的东西最后才能取出；最后放入的东西最先取出（先进后出）。

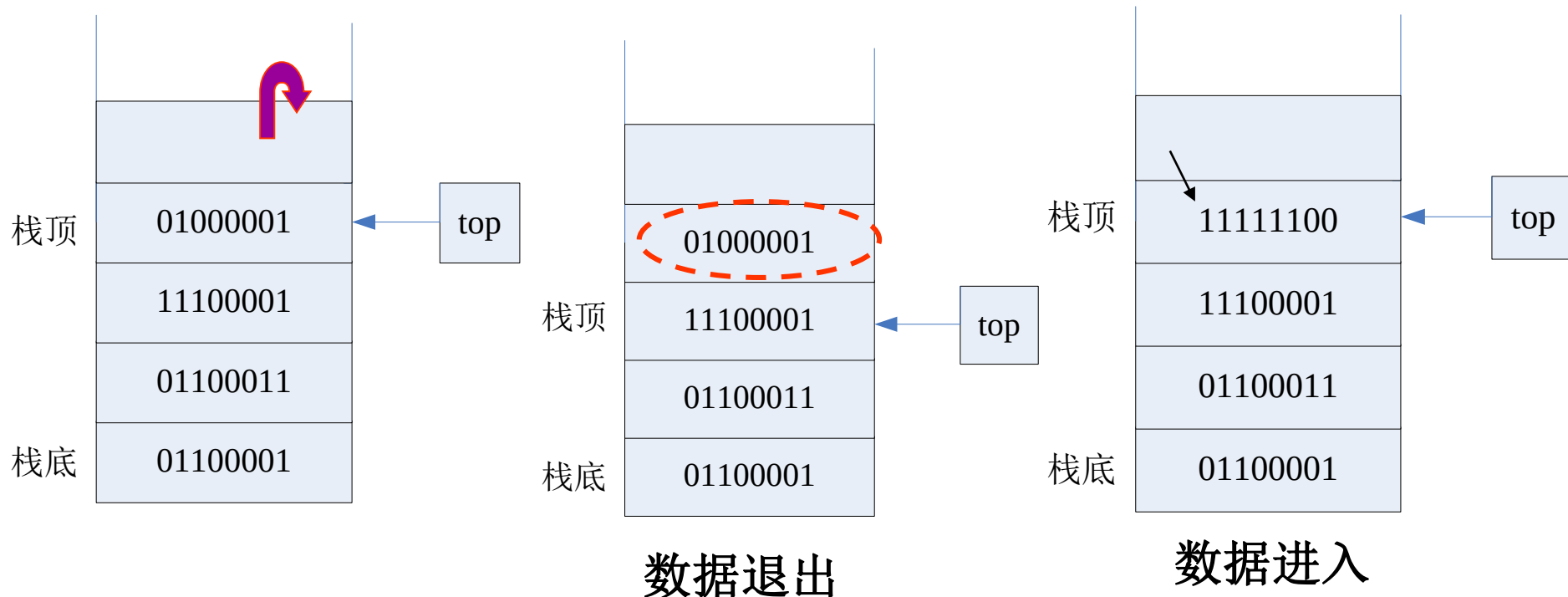
## 5.4.3 函数的调用—栈区简介

- 栈区
- 一片存放用户数据的内存空间；一端“封”了口（栈底），一端“开”着口(栈顶)；
- 保存数据：数据只能从顶部（栈顶）进入；
- 取数据：栈顶的数据先被取出，栈底的数据最后被取出。即数据是“先进后出”。数据的进入和退出均在栈顶进行。
- 读数据：只能读取  
栈顶的数据



## 5.4.3 函数的调用—栈区简介

设计了一个位置指示`top`，用来指示当前的栈顶位置。  
通过`top`可以访问当前栈顶数据。数据的进入和退出通过修改`top`的值来实现。

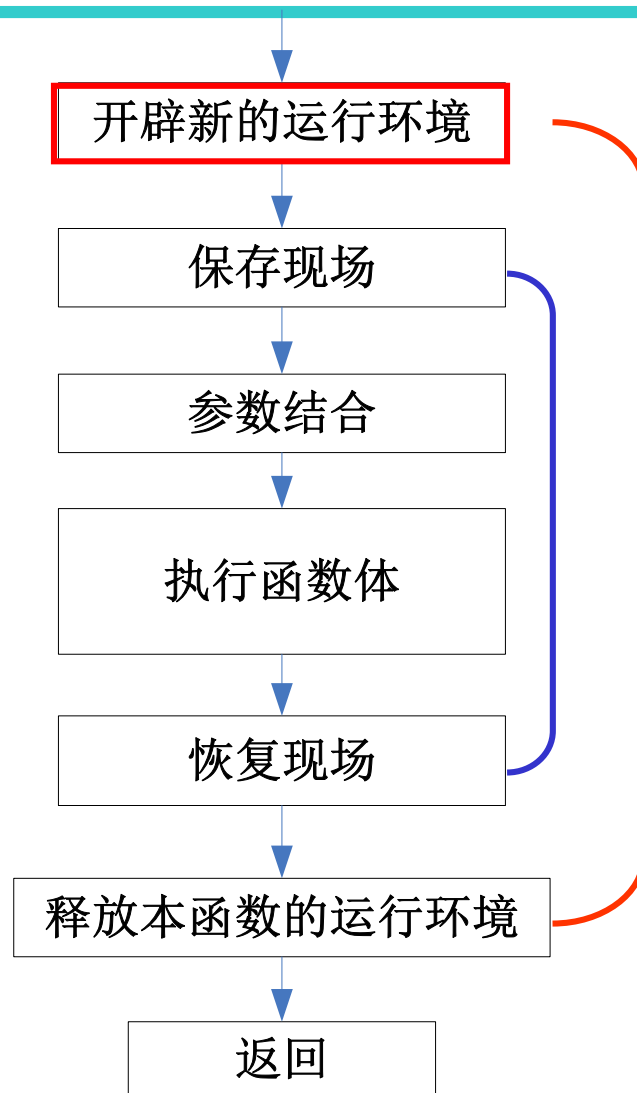


# 函数调用过程-开辟新的运行环境

运行环境即指函数执行时需要的数据空间。需要哪些数据空间？

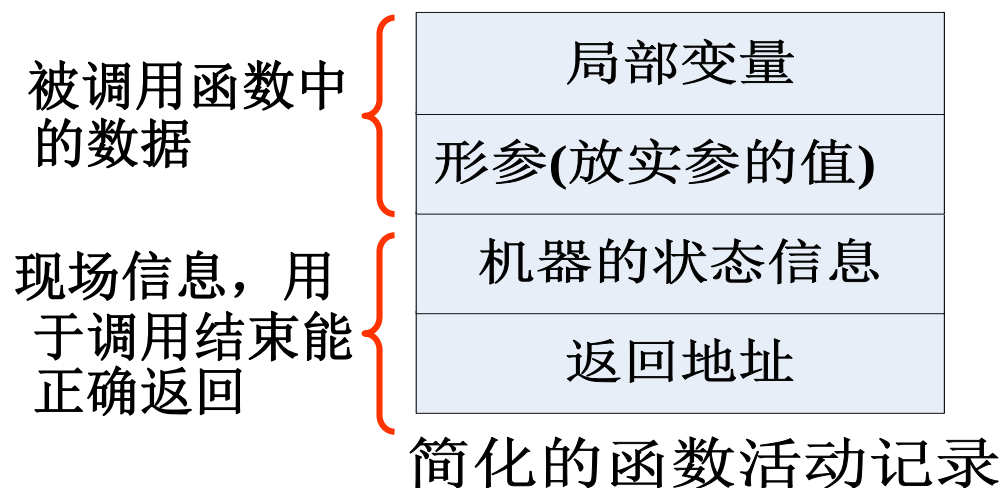
函数执行时需要的数据空间

1. 生存期在本次函数执行过程中的数据对象，如形参、局部变量等；
2. 用以管理函数调用过程的信息。当函数A调用函数B时，函数A的运行被中断，当前机器的状态信息，如程序计数器（返回地址）、寄存器的值等都必须保存，以便调用结束后，能准确返回到函数A并继续正确执行。



## 5.4.3 函数调用过程-开辟新的运行环境

- 为方便，引入一个术语：“**函数的活动记录**”。函数的活动记录是一段在**栈区**分配的连续的内存存储区，用以存放函数一次执行所需的数据。
- 开辟新的运行环境即指在**栈区的栈顶**创建一个函数活动记录。释放本函数的运行环境即指从栈顶将活动记录释放。





## 5.4.3 函数调用过程-开辟新的运行环境

```
1.  int square(int);  
2.  int main()  
3.  {  
4.      int x;  
5.      for (x=1; x<=10; x++)  
6.          printf("%4d",square(x));  
7.      return 0 ;  
8.  }  
9.  int square(int y) /*函数定义*/  
10. {  
11.     return(y*y);  
12. }
```

√

|           |
|-----------|
| 局部变量      |
| 形参(放实参的值) |
| 机器的状态信息   |
| 返回地址      |

√

简化的函数活动记录

为简化起见，假设  
square函数的活动  
记录只包含分配给  
形参y和返回地址的  
存储空间

图0 操作系统调用  
main函数前

栈底

x  
返址

|    |  |
|----|--|
| x  |  |
| 返址 |  |

栈底

x  
返址

操作系统

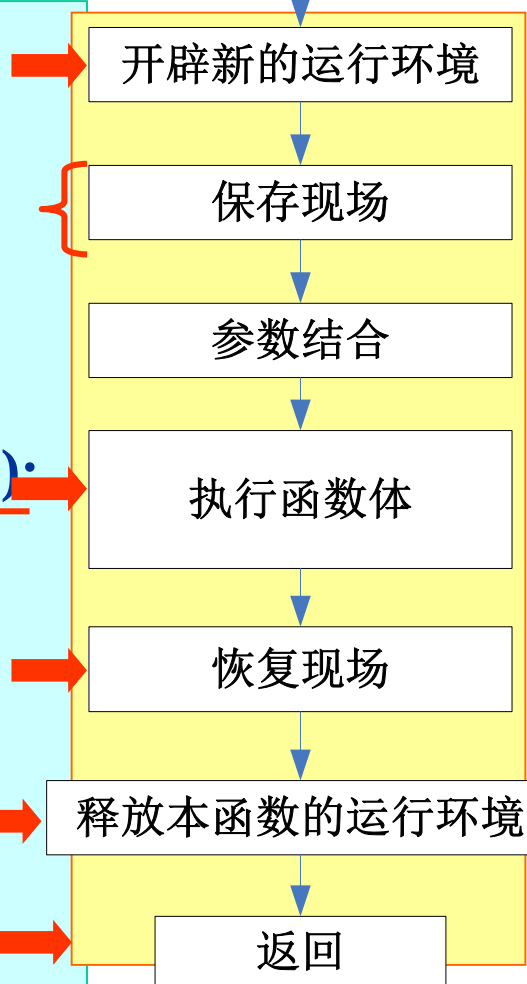
|    |      |
|----|------|
| x  |      |
| 返址 | 操作系统 |

图1 运行main函数

图2 main函数x赋值为1

```

1. int square(int);
2. int main()
3. {
4.     int x;
5.     for (x=1; x<=3; x++)
6.         printf("%4d",square(x));
7.     return 0 ;
8. }
9. int square(int y) /*函数定义*/
10. {
11.     return(y*y);
12. }
    
```



|    |       |
|----|-------|
| y  | 1     |
| 返址 | /*6*/ |
| x  | 1     |
| 返址 | 操作系统  |

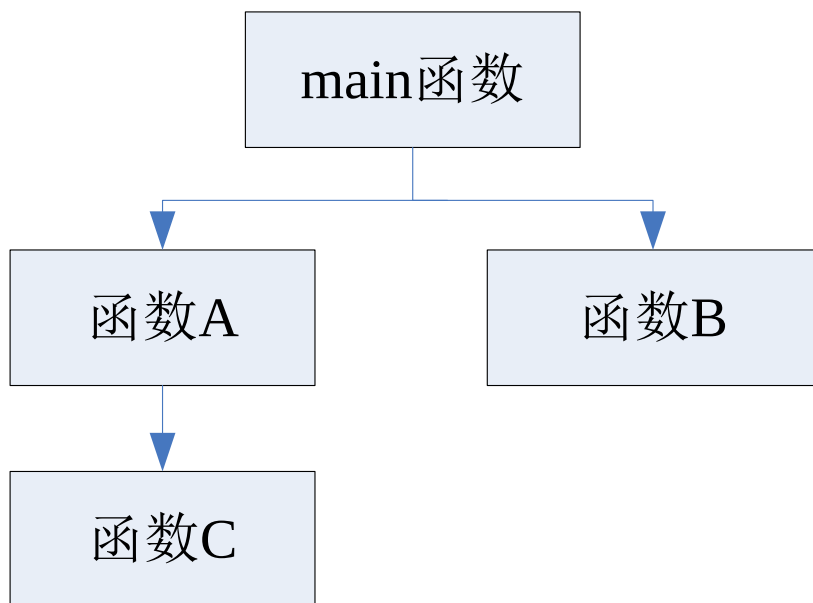
图3 square函数活动记录入栈

|    |      |
|----|------|
| x  | 1    |
| 返址 | 操作系统 |

图4 square函数活动记录出栈

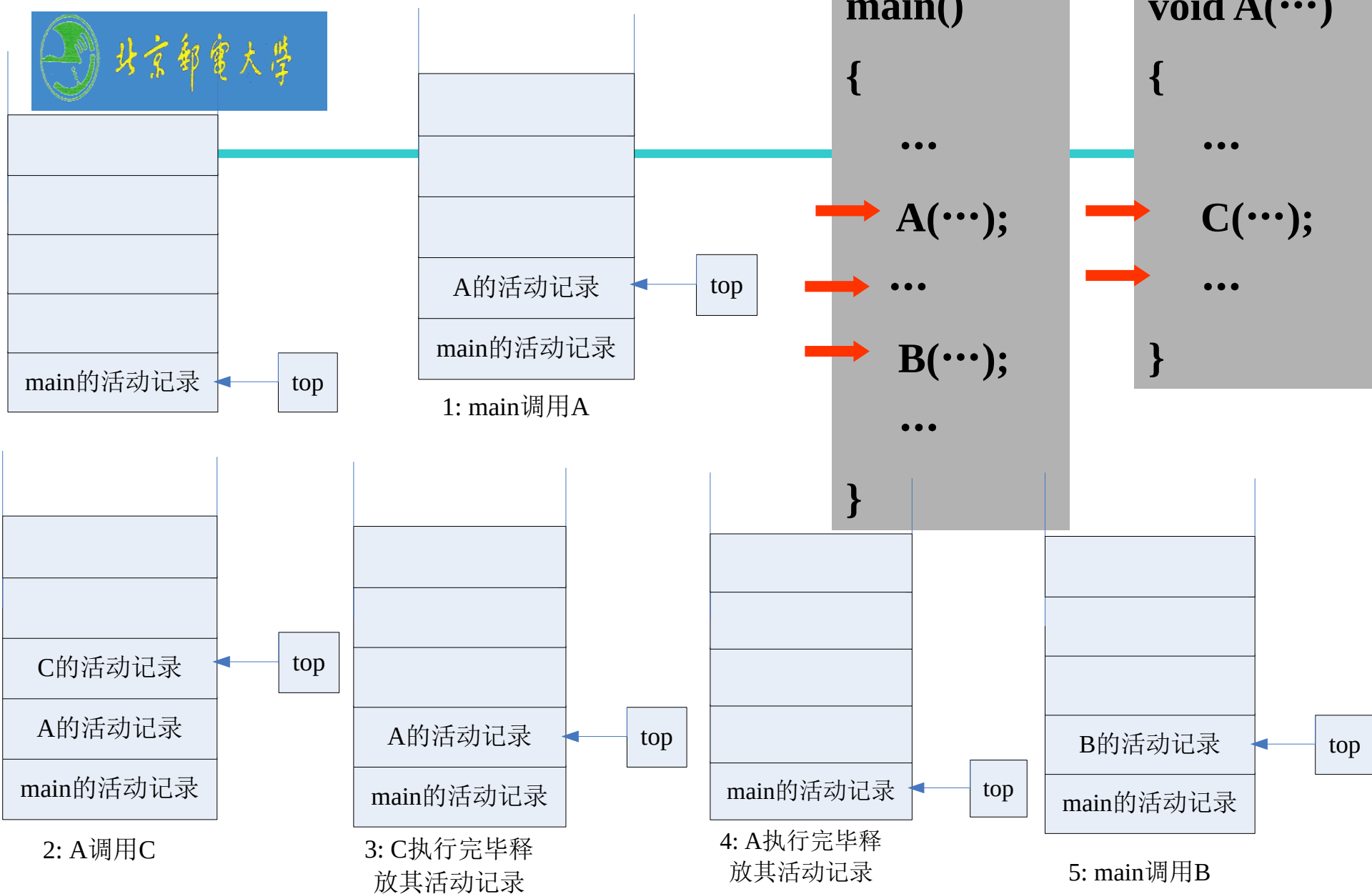
若把main中的变量改名为y，则由于其与square中的y占用不同内存，因此不会相互影响。

# 函数活动记录使用示例



```
main()
{
    ...
    A(...);
    ...
    B(...);
    ...
}
```

```
void A(...)
{
    ...
    C(...);
    ...
}
```



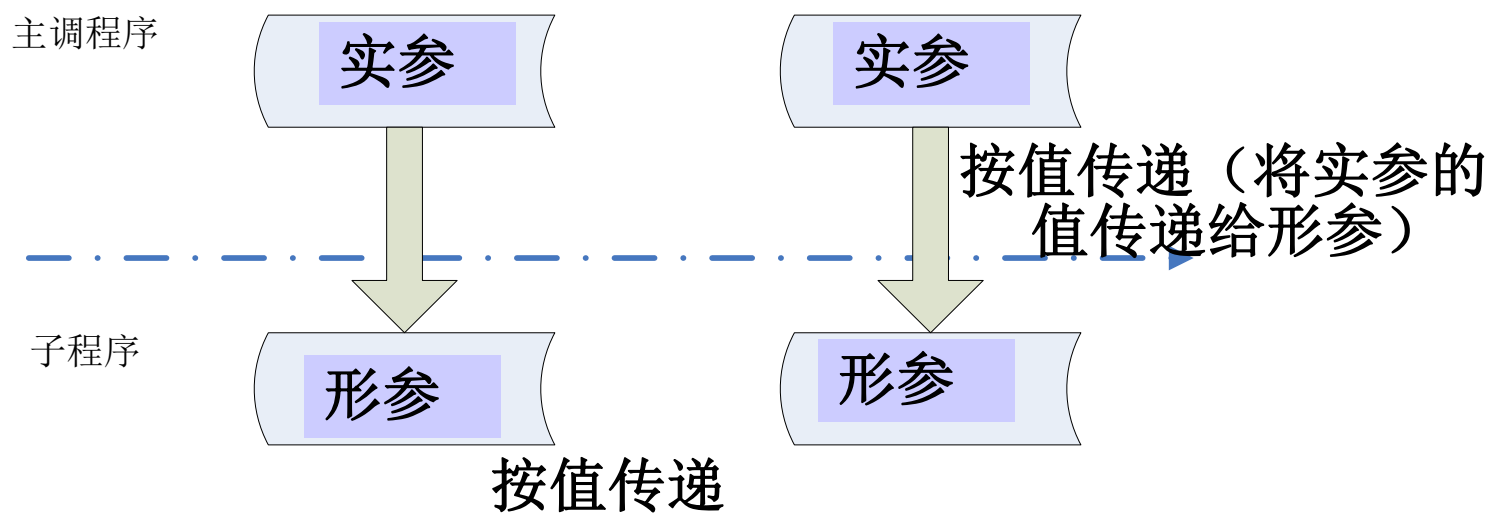
## 总结

- 如何确保能够逐层返回到上一级调用？  
函数A调用函数B，则在函数B的活动记录中记录了A的返回地址。返回前取出该地址，即能正确返回。
- 为何不同的函数可以使用同名的参数和变量？  
因为不同函数的活动记录占用不同的内存单元，程序运行时始终是从位于栈顶的活动记录中取形参和变量的值。

## 5.4.3 函数的调用

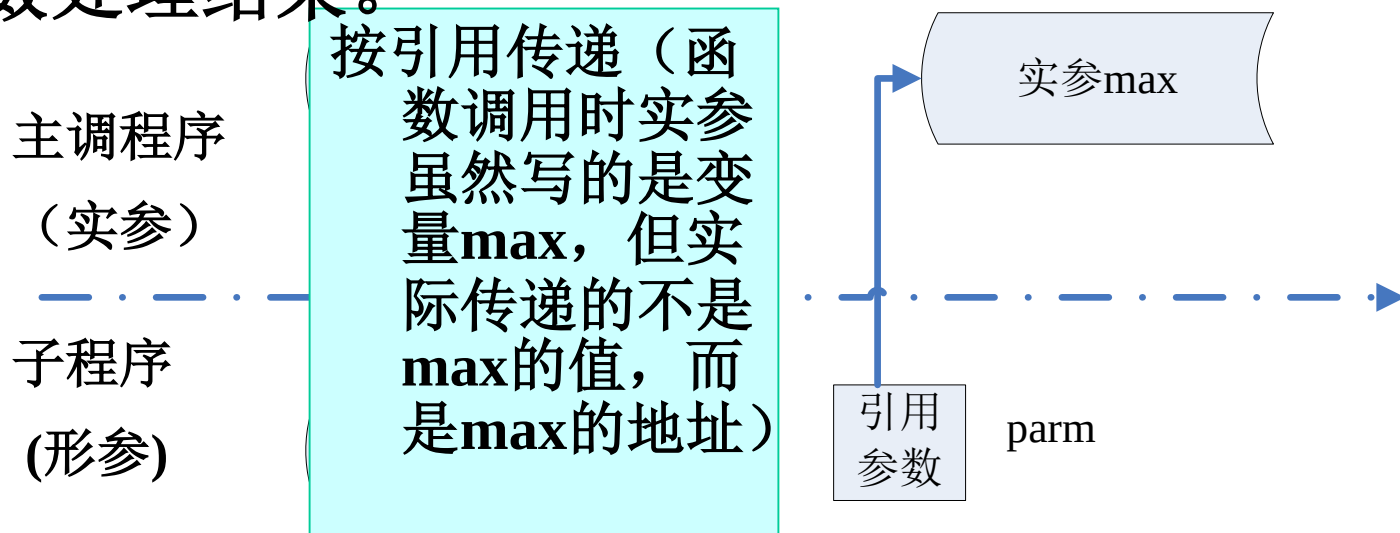
子程序参数传递两种方式：按值传递和按引用传递。

**按值传递：**实参的值被复制并置入被调用子程序的形参中。此方式下不管在被调用子程序中怎样操作并改变形参的值，在主调程序中的实参的值都是安全的未发生变化的。



## 5.4.3 函数的调用

- 如何解决函数的多返回值问题？
- 按引用传递：此时实参必须是变量，子程序调用时将**实参的地址**而不是实参的值置入被调子程序的形参中。被调子程序对形参的操作实际上是对主调程序中实参的操作，从而向主调程序传回函数处理结果。



## 5.4.3 函数的调用

注意：C语言中所有的调用都是**传值调用**，但是可以通过其他方式来实现函数的多返回值（即实参本身存放的就是某个变量的内存地址，将该地址传递给被调用函数后，被调用函数通过该地址来访问变量，将函数处理结果写入变量）。具体以后学习指针时讲解



## 5.3 子程序概念与设计

## 5.4 C语言的函数

### 5.4.1 函数

### 5.4.2 函数的定义

### 5.4.3 函数的调用

### 5.4.4 函数原型

## 5.5 头文件

## 5.6 函数应用举例

```
float square(float ); /*函数原型*/
```

## 5.4.4 函数的原型

- (1) C语言的原则（先声明、后使用）；
- (2) 函数原型的作用：函数原型是对被调用函数的**接口声明**，它告诉编译器函数返回的数据类型、函数所要接收的参数个数、参数类型和参数顺序，编译器用函数原型校验函数调用是否正确。
- (3) 函数原型一般格式：  
返回值类型 函数名(数据类型[ 参数名1], 数据类型[ 参数名2]...); 注意：参数名可写可不写。

## 5.4.4 函数的原型

(4) 函数原型可以放置在任何函数之外，出现在该函数原型之后的所有函数都可以调用对应的函数；也可以放在某个函数中，此时只有该函数能够调用它。从程序设计风格考虑，最好是将函数原型集中在一起，放在主函数之前。



**/\*square函数在main和fun中  
均可被调用\*/**

**float square(float);**

**int main()**

**{**

**...**

**}**

**int fun()**

**{...**

**}**

**float square(float y)**

**{**

**return(y\*y);**

**}**

**int main()**

**{**

**/\*square函数只能在  
main中被调用\*/**

**float square(float);**

**...**

**}**

**int fun()**

**{...**

**}**

**float square(float y)**

**{**

**return(y\*y);**

**}**

## 5.4.4 函数的原型

(5) 当被调用函数的函数定义出现在该函数调用之前时，可以省略函数原型。因为在调用之前，编译系统已经知道了被调用函数的函数类型、参数个数、类型和顺序。

```
/*函数定义*/  
float square(float y)  
{  
    return(y*y);  
}  
int main()  
{  
    ...  
}
```

## 5.4.4 函数的原型

(6) 函数原型、函数定义、函数调用要保持一致。和函数原型不匹配的函数调用会导致语法错误；函数原型和函数定义不一致，也会产生错误。

```
#include<stdio.h>
float square(float y);//函数原型
int main()
{
    float x=2.5;

    printf(“%.2f”,square(x)); //函数调用
}
/*函数定义*/
float square(float y)
{
    return(y*y);
}
```

## 5.4.4 函数的原型

(7) 如果程序中没有包含函数原型，则编译器就会用第一次出现的该函数（函数定义或函数调用）来构造函数原型。默认情况下，编译器假定函数的返回类型为`int`类型，而对参数不作任何假定。

思考：编译器为何不会提示printf未定义呢？

在文件stdio.h中已经给出了函数原型

## 函数的原型

```
#include<stdio.h>
```

```
float square(float); //形参名可写可不写
```

```
int main()
```

```
{
```

```
    float x=2.5;
```

```
    printf(" %.2f",square(x));
```

```
    return 0 ;
```

```
}
```

```
/*函数定义*/
```

```
float square(float y)
```

```
{
```

```
    return(y*y);
```

```
}
```

编译到蓝色行时报错：  
conflicting types for  
square。

错误原因：当自上而下对  
源程序编译时，编译到  
红色字体那一行，编译器  
会默认square(x)函数  
返回值类型是int型，从  
而和后面的float冲突。

解决方法：在函数调用前  
使用函数原型对函数进  
行声明。



## 5.4.4 函数的原型

### (8) 函数原型可以用来强制转换参数类型。

在函数调用之前，和函数原型中的参数类型不完全一致的实参值会被转换为合适的类型。

如： `sqrt` 的参数类型是 `double`，进行如下调用 `printf(“%.3f\n”,sqrt(4))` 时，在将4传递给 `sqrt` 之前先转换为 `double` 类型的值4.0；

参数类型的转换有可能是低类型向高类型转换，也可能是高类型向低类型转换。高类型向低类型转换可能会导致不正确的结果（如 `long` 类型向 `short` 类型的转换）。

5.3 子程序概念及设计

5.4 C语言的函数

**5.5 头文件**

5.6 函数应用举例



- 对于一些通用的函数（如输入输出函数、数学函数等），可能在不同的程序中都会用到。
- 为了使用这些函数，需要在程序中说明其函数原型。
  - 一种方式是在程序中逐个写出函数原型；
    - `double sqrt(double x) ;`
    - `double fabs(double x) ;`
  - 另一种方式是将这些函数原型集中在一起，形成.h头文件,然后在程序中直接包含这些头文件。
    - `#include<math.h>`

每一个标准库都有一个相应的头文件,文件扩展名为.h (如**stdio.h**, [示例](#))。该头文件包含了该库中所有函数的原型以及这些函数所需的所有常量和数据类型的定义。

程序员可以根据需要自己建立头文件,使用**include**命令可以把程序员定义的头文件包含到程序中,如:

```
#include "square.h"
```

*注意:*

**#include<stdio.h>**包含标准库的头文件

**#include "square.h"** 包含程序员自定义的头文件

5.3 子程序概念及设计

5.4 C语言的函数

5.5 头文件

**5.6 函数应用举例**



### 例1、验证歌德巴赫猜想

任一充分大的偶数，可以用两个素数之和表示，  
例如：

$$4 = 2 + 2$$

$$6 = 3 + 3$$

$$10 = 3 + 7$$

$$10 = 5 + 5$$

..

$$98 = 19 + 79$$

## 例1、验证歌德巴赫猜想

- 思路：偶数num是要分解的数，则
- $\text{num} = i + (\text{num} - i)$
- 其中i和 $(\text{num} - i)$ 都得是素数
- 因此可以对i可能的取值进行穷举。

## 例1、验证歌德巴赫猜想

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int num;/*num:要判断的一个偶数*/
```

```
    int num1; /*num表示为两个素数num1和num-num1之和*/
```

```
    int count;/*计数输出个数，用于换行。*/
```

```
    printf("输入要验证的偶数: ");
```

```
    scanf("%d",&num);
```

```
    if (num%2!=0)
```

```
        printf("输入的数不是偶数，程序终止\n");
```



## 例1、验证歌德巴赫猜想

```
else{//采用穷举法，将num分解为两个素数之和
    count=0;
    for(num1 = 2;num1 <= num/2;num1++){
        if (isPrim(num1) && isPrim(num-num1)){
            printf("%d = %d + %d\t",num,num1,num-num1);
            count++;
            if(count%3 == 0)
                printf("\n");//每输出3个数换一行
        } //end of if
    } //end of for
} //end of else

return 0;
}
```

## 5.6 函数应用举例

- 例2：设计一个函数，将十进制数转换成二进制、八进制和十六进制。然后在主函数中读入一个整数，调用函数，输出转换结果。
- 算法思路：

## 例2.进制转换

- 假设将十进制数57转换为二进制
- 从右到左写出每列的位值，直到发现位值大于该十进制数的列。这样就先得到  
位值：64 32 16 8 4 2 1
- 然后去掉位值为64的列，得到：  
位值：32 16 8 4 2 1
- 然后，从左至右进行。57除以32得商为1，余数为25，所以在32这列写下1，然后25除以16商为1，余数为9，所以在16这列写下1，……  
位值： 32 16 8 4 2 1  
符号值： 1 1 1 0 0 1 所以 $(57)_{10}=(111001)_2$

## 例2.进制转换

- 假设将十进制数57转换为八进制
- 从右到左写出每列的位值，直到发现位值大于该十进制数的列。这样就先得到  
位值：64 8 1
- 然后去掉位值为64的列，得到：  
位值：8 1
- 然后，从左至右进行。57除以8得商为7，余数为1，所以在8这列写下1，然后1除以1商为1，余数为0，所以在1这列写下1。  
位值：8 1  
符号值：7 1 所以 $(57)_{10}=(71)_8$

如何设计一个通用的函数？参数如何设计？返回值如何设计？

## 例2.进制转换

**void transfer(int num,int base)**

输入十进制数到num中

计算得到p(p为base的x次方, 且  
是大于num的最小数)

进行进制转换并输出

**p = 1**

**p <= num**

**p = p \* base**

**p = p / base**

**p != 0**

**k = num / p**

输出

**num = num % p**

**p = p / base**

## 例2.进制转换

```
/*将num转换为base进制并输出*/  
void transfer(int num,int base)  
{  
    int p,k;  
    p = 1;  
    while (p <= num) //求p: p是base的x次幂, 且p大于num  
        p = p * base;  
  
    p = p / base;  
  
    /*循环求base进制数的各位*/  
    while( p != 0){  
        k = num / p; /*计算当前要输出的那个base进制数*/  
        if (k <= 9)  
            printf("%d",k);  
        else  
            printf("%c",k - 10 + 'A');  
  
        num = num % p;  
        p = p / base;  
    }  
}
```

## 例3：掷骰子游戏（函数调用的练习）

### 例3：掷骰子游戏

#### 1、rand()函数：

产生一个0到RAND\_MAX之间的整数，使用时需要包含头文件<stdlib.h>。

RAND\_MAX 是在头文件<stdlib.h>中定义的符号常量，ANSI规定其不得小于32767。

用于产生1~6之间的随机数： **$1 + \text{rand()} \% 6$**

## 例3：掷骰子游戏

- 每一次调用函数rand()时，0到RAND\_MAX之间的每一个数字被选中的机会几乎均等（见下页）
- rand()所产生的随机数是伪随机数，反复调用rand()产生的一系列数似乎是随机的，但是每次执行程序所产生的序列则是重复的。



```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    int i;
    for(i=1;i<=20;i++)
    {
        printf("%10d",1+rand()%6);
        if(i%5==0)
            printf("\n");
    }

    return 0;
}
```

```
6      6      5      5      6
5      1      1      5      3
6      6      2      4      2
6      2      3      4      1
```

```
6      6      5      5      6
5      1      1      5      3
6      6      2      4      2
6      2      3      4      1
```

### 2、srand(unsigned int)函数

如何实现真正的随机化：为函数rand设置**随机数种子**；每次执行程序时，只要随机数种子不同，产生的随机数序列也将不同（见下页）。

让机器设置随机数种子：

**srand(time(NULL))**:通过time函数使计算机读取当前时间值（以秒为单位，如1099228431），并把该值设成随机数种子，这样可以避免用户自己输入随机数种子。

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    int i;
    int seed;

    printf("请输入种子: ");
    scanf("%d",&seed);
    srand(seed);
    for(i=1;i<=20;i++)
    {
        printf("%10d",1+rand()%6);
        if(i%5==0)
            printf("\n");
    }
    return 0;
}
```

请输入种子: 67

|   |   |   |   |   |
|---|---|---|---|---|
| 6 | 1 | 4 | 6 | 2 |
| 1 | 6 | 1 | 6 | 4 |
| 2 | 3 | 6 | 2 | 1 |
| 4 | 1 | 4 | 4 | 6 |

请输入种子: 56

|   |   |   |   |   |
|---|---|---|---|---|
| 6 | 1 | 4 | 3 | 4 |
| 3 | 1 | 3 | 4 | 3 |
| 3 | 4 | 4 | 6 | 1 |
| 5 | 5 | 6 | 2 | 6 |

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>
```

```
int main()
{
```

```
    int i;
```

```
    srand(time(NULL));
```

```
    for(i=1;i<=20;i++)
```

```
    {
```

```
        printf("%10d",1+rand()%6);
```

```
        if(i%5==0)
```

```
            printf("\n");
```

```
    }
```

```
    return 0;
```

```
}
```

```
        6         5         6         4         5
        2         1         5         6         1
        6         1         1         4         5
        3         1         4         5         3
```

```
        4         2         2         1         2
        6         4         5         5         6
        2         1         4         5         4
        3         3         3         3         5
```

### 例4、碰运气游戏

游戏规则：投掷两个骰子，

- 1、第一次投掷时，如果所得和为7或者11，则游戏者赢了；如果得到的和为2、3或者12，则游戏者输了；
- 2、如果和为4、5、6、8、9或10，那么这个和就是游戏者的点数，要想赢的话，必须继续投掷骰子，直到取得自己的点数为止，但是，如果投掷出了点数7，那么游戏者就输了。

|        |
|--------|
| 設置種子   |
| 第一次擲骰子 |
| 遊戲未結束  |
| 繼續投擲骰子 |
| 輸出結果   |



逐步求精

1、將投擲骰子設計成函數

**int rollDice (void)**

返回兩個骰子的面值

2、定義一個變量表示遊戲是否結束

**int gameStatus**

```
srand(time(NULL));  
sum=rollDice(); //第一次投掷骰子  
switch(sum) {  
    case 7:case 11: gameStatus=1; break;  
    case 2: case 3: case 12: gameStatus=0; break;  
    default:  
        gameStatus=2; myPoint=sum;  
        printf("my point is: %d\n",myPoint);break;  
}  
while(gameStatus==2){ //继续投掷骰子  
    sum=rollDice();  
    if (sum==myPoint)  
        gameStatus=1;  
    else  
        if (sum==7) gameStatus=0;  
}  
if (gameStatus) //输出结果  
    printf("游戏者赢了! \n");  
else  
    printf("游戏者输了! \n");  
return 0;  
}
```

习题目的：

- 1、温习随机数的产生方法；
- 2、switch结构、if-else结构、while结构综合运用；
- 3、体会使用函数的好处（代码复用、降低代码书写难度、提高代码易读性...）；
- 4、体会gameStatus变量对简化程序结构的作用；



## 例5. 打印日历

例5. 打印日历：输入年份和月份，输出该月日历。  
如下图所示：

```
F:\70、WorkingFolder-My\上课讲义\计算机导论与程序设计(06~07)\4、典型例题(C语言)
input the year and month<YYYY-MM>:2006-12
2006-12
Sun.    Mon.    Tue.    Wed.    Thu.    Fri.    Sat.
        1      2
3        4      5      6      7      8      9
10       11     12     13     14     15     16
17       18     19     20     21     22     23
24       25     26     27     28     29     30
31
请按任意键继续 . . .
```

## 例5. 打印日历

输入年`year`和月`month`

求出`year`年`month`月的1号是周几

求出`year`年`month`月的天数

打印`year`年`month`月的日历

问题的关键是求`year`年`month`月的1号是周几。

假设0001年年初到第`year`年`month`月的1号总天数为`days`天，则  
可通过下述公式判断第`days`天是周几：

$\text{weekDay} = \text{days} \% 7$

(`weekDay`值为0，表示周日。`WeekDay`为1~6，分别表示周一到周六)

## 例5. 打印日历

- 那么，如何求得0001年年初到第year年month月1号的总天数days天？

可先通过下述公式求得0001年年初到第year-1年年底的总天数

$$\text{days} = (\text{year}-1) * 365 + (\text{year}-1) / 400 + (\text{year}-1) / 4 - (\text{year}-1) / 100$$

然后，再求出year年1月到month-1月的天数yearDays。

**days** + yearDays即为所求结果。

## 例5. 打印日历

1. 假设公元0001年年初到第year年年底的总天数为days天, 则

$$\text{days} = \text{year} * 365 + \text{year} / 400 + \text{year} / 4 - \text{year} / 100$$

分析如下:  $\text{days} = \text{year} * 365 +$  0001年到year年闰年总年数

闰年条件: 能被4整除但不能被100整除 或者 能被400整除。

能被4整除但不能被100整除的年数:

设0001年到year年年底 能被4整除的年数为x(则  $x = \text{year} / 4$ )

0001年到year年年底 能被4整除但不能被100整除的年数为 y,

0001年到year年年底 能被4整除也能被100整除的年数为 z

(则  $z = \text{year} / 100$ , 因为能被100整除则肯定能被4整除)

$$\text{则 } y = x - z = \text{year} / 4 - \text{year} / 100$$

能被400整除的年数为:  $\text{year} / 400$

故0001到year年内闰年总年数为:

$$\text{year} / 400 + \text{year} / 4 - \text{year} / 100$$

2. 判断第days天是星期几:

$\text{weekDay} = \text{days} \% 7$

( weekDay值为0, 表示周日。WeekDay为1~6, 分别表示周一到周六)

**int GetWeekDay(int year,int month,int day)**

函数功能：求year年month月day日是星期几。

参数：年、月、日。

返回值：0~6。其中0代表周日，1~6代表周一到周六。

**int GetMonthDays(int year,int month)**

函数功能：求year年month月共有几天。

参数：要求天数的年和月。

返回值：该月的天数。

## 例5. 打印日历

**void PrintMonthCalender(int startDay,int days)**

函数功能：打印某月日历。

参数：startDay表示该月1号是星期几。startDay取值为0~6，其中0代表周日，1~6代表周一到周六。days：该月共有几天。

返回值：无。

**int CheckLeap(int year)**

函数功能：检测是否闰年。

参数：year：要检测的年。

返回值：0或者1。0：不是闰年；1：是闰年。

- 定义：将常用的功能函数集中起来，经过编译形成机器二进制指令并存放在一个库文件内的函数集被称为库函数。库函数通常由编译器制造商提供，以头文件与库文件一一对应的方式供程序员编程调用。
- 引用规则：
  - 静态：使用预处理语句`include`引用头文件的方式实现对库函数的调用；
  - 动态：使用加载函数在程序运行中动态调用库函数的；



- 数学库：数值计算类；
- 字符（串）处理库：字符串类；
- 系统访问库：访问操作系统内部功能类；
- 数据转换库：变换数据类型；
- 动态内存管理库：随机内存管理类；
- 图形库：显示图形类；
- 统计学库：统计及相关图表类；
- 外来库：第三方软件类；

- 数学库：分为系统级与专业级。
  - 系统级：常用数学函数被置于stdio.h的引用范围内，如三角函数、常用对数等；
  - 示例：
    - 对数函数： `double log(double);`
    - 正弦函数： `double sin(double);`
    - 求平方根函数： `double sqrt(double);`
    - 随机函数： `void srand(unsigned int);`
  - 专业级：使用math.h的引用范围内，如定积分、多次方程解等；

- 字符串库：
  - 字符串复制: `char* strcpy(char*,char*);`
  - 求字符串长度: `unsigned int strlen(char*);`
  - 匹配字符串内的第一个字符: `char* strchr(char*,char);`
- 系统访问库：
  - 计算机时间: 计算机主板上的时间实际是自格林威治时间1970年1月1日0时起的时间按秒计数的, 程序获取后要经过一系列的计算后分别转换为年、月、日、时、分、秒等数据。时间库头文件均在`time.h`内声明。

//获取计算机时间示例:

```
#include<stdio.h>
```

```
#include<time.h>
```

```
int main()
```

```
{
```

```
    struct tm* maclocaltimer;//分解多维时间计量单位的时间结构体
```

```
    time_t mactime=time(NULL);//获得自格林威治时间1970年1月1日  
    0时起的时间计数
```

```
    maclocaltimer=localtime(&mactime);//将本机时间分解为多维时间  
    结构
```

```
    printf( "current time is %ld\n",mactime);
```

```
    printf(" %d\n",maclocaltimer->tm_year+1900);
```

```
    return 0;
```

```
}
```

**struct tm//多维时间结构体**

```
{  
    int tm_sec;    /* Seconds */  
    int tm_min;    /* Minutes */  
    int tm_hour;    /* Hour (0--23) */  
    int tm_mday;    /* Day of month (1--31) */  
    int tm_mon;    /* Month (0--11) */  
    int tm_year;    /* Year (calendar year minus 1900) */  
    int tm_wday;    /* Weekday (0--6; Sunday = 0) */  
    int tm_yday;    /* Day of year (0--365) */  
    int tm_isdst;    /* 0 if daylight savings time is not in effect) */  
};
```

